The application and the demo

API → page states → comparison → scenarios → live demo

What you need to know

- What an API is here: a small set of web addresses (endpoints) that return prepared results as JSON — plain text data that a browser can read. The browser asks, the server answers with a few hundred rows, never with the millions of raw rows in the database.
- Why that matters: the SQL, the validation and the credentials stay on the server. The page only displays what the server already prepared and checked.
- The numbers of your own demo pair, so you never read them off the screen for the first time while presenting: in Serie A 24/25 Inter recorded 79 GF, 35 GA and +44 GD (attack rank 1, defense rank 2) and Milan 61, 43 and +18 (ranks 3 and 7); their two meetings ended 1–2 and 1–1, so the head-to-head total is 2–3.
- The three states that are easy to confuse: no coverage for this league and season, coverage exists but the two clubs never met, and a real completed 0–0 draw. The interface shows a different message for each and never invents a score.
- What the HTTP status codes mean in our app: 422 means the input is invalid, for example the same club selected twice; 404 means the requested club or season is not in scope; 503 means the data or the model is not ready yet, which is a readiness problem, not a bad request.
- How head-to-head goals are counted: for each completed meeting we take the selected club's perspective, so its goals are the home score when it played at home and the away score otherwise. Swapping the two clubs swaps the totals. Wins, draws and points follow from those scores.
- What protects the Lab inputs: the server rejects values that are not finite numbers, checks the allowed range and refuses an attacking budget larger than the total budget. The sliders also limit the range, but the server check is the one that matters, because anyone can call the API directly.
- Every new field reads from a table only after checking that the table exists. If the optional match data was never loaded, the API answers “absent” instead of failing — that is why the demo survives a partial setup.
- What the automated checks cover: 43 unit tests, 16 live API checks and 15 real-browser checks, including mobile layout at 390 pixels. They are scoped evidence, not proof that no bug exists.

What to show

1. Open /api/docs and call one endpoint live, then show the same numbers on the page: the page is a view of the API, not a separate truth.
2. Open a comparison for two clubs in a season with match data, then switch to a league-season without it and read the coverage message aloud.
3. In the Lab, move one input, show the new prediction, then press Reset and explain that a slider changes a model input, not reality.

Your spoken part

> *The browser reads a bounded API over prepared results. League and season filters are explicit, and the URL preserves the selected scope. In the comparison, season totals and actual head-to-head meetings are separate. Serie A 24/25 shows Inter at 79 for and 35 against (+44), ranked first in goals scored per match and second in goals conceded per match. Milan has 61 for and 43 against (+18), ranked third and seventh. The source has no xG field. For Inter and Milan in 24/25, the recorded league scores are 1–2 and 1–1, giving head-to-head goals of two to three. Missing match enrichment is reported explicitly. The Lab loads the five real club-season inputs, calls a saved model and rejects invalid budgets or non-finite numbers. We verify behavior at three levels: regression tests, live data contracts and desktop/mobile browser checks.*

Hand over: “The final question is whether this system runs reproducibly and what resources it uses.”

Your slots on the clock

Slide	Block	Time	Length
6. See the evidence	06 / Live demo	06:30–09:15	165 s
10. Boundaries and ownership	10 / Close	14:00–15:00	60 s

How to close the exam

1. You own slide 6 (the live demo, 06:30–09:15) and slide 10 (the close, 14:00–15:00). The demo is the longest single block and the most fragile one.
2. Follow the five demo steps in order and say what you expect to appear before it appears. That turns a loading delay into a prediction instead of an awkward silence.
3. If something is not up, do not debug Docker in front of the examiner. One sentence — “we move to the recorded evidence” — then use the screenshots in defense/site/assets.
4. Show the missing-coverage state on purpose. It proves the interface does not invent a 0–0 result.
5. In the Lab change exactly one input, then reset. One factor at a time is the same discipline we use in the measurements.
6. If asked why an API instead of plotting the tables directly: millions of rows never travel to the browser, and SQL plus validation stay on the server.
7. You speak last. Name the project's boundaries and the open items yourself — the examiner hears honesty as strength, not as weakness.

Not your part — hand it over

1. Infrastructure numbers are not yours. Startup time, memory, the index speed-up and the growth experiment belong to Atai — hand them over in one sentence.
2. Model quality is not yours either. R^2, MAE, the holdout and the baseline belong to Lilia. You own what appears on the screen and how the API behaves.
3. A safe and accurate answer when a number comes up during your demo: “the page shows what the API prepared; the measurement behind it is Atai's part.” Then continue the demo.
4. If you are asked for a number that is not in this guide, say where it is written instead of guessing: docs/findings.md for results, docs/MEASUREMENTS.md for timings.

Where to look in the project

Project file	What to look at in it
`app/api.py`	Endpoint contracts, the 422 / 404 / 503 answers, bound parameters, read-only transactions and the table-existence guard behind every new field.
`app/static/app.js`	The page states: missing coverage, no recorded meeting and a real 0–0 are three different outcomes.
`app/static/api-docs.html`	The API documentation page you open during the demo at /api/docs.
`scripts/check_live.py`	The 16 live checks of the API contracts and the prepared outputs; run it after the stack is healthy.
`scripts/browser_check.mjs`	The 15 real-browser checks of actual flows and the mobile layout; screenshots land in artifacts/.
`scripts/site_check.mjs`	The nine checks of the offline study site; the result is written to defense/site-check.json.
`tests/test_head_to_head.py`	Proof that goals for and goals against swap when the two compared clubs are reversed.
`tests/test_health.py`	Why 503 means “data or model not ready” rather than “bad request”.
`docs/FRONTEND_HANDOFF.md`	The data contract for any other frontend — evidence that presentation is separated from processing.

Plain-language glossary

These definitions are deliberately non-technical. Use them to explain the system to someone outside the course, and to answer a question whose wording you do not recognise.

Container / Docker Compose	A container is one packaged program with everything it needs to run. Compose is the file that starts our four containers in the right order: the database first, then the loader, then the processing pipeline, then the web application.
Relational database / PostgreSQL	A store of tables with fixed columns that can be combined by shared keys. We use PostgreSQL 16. Combining two tables on a shared key is called a join.
Schema	The declared shape of the tables: which columns exist and what type each one holds. It is written in sql/schema.sql.
COPY and index	COPY loads a whole file into a table in one stream. An index is a lookup structure, like the index of a book: it makes selective queries fast and is built after the bulk load.
Pipeline	The processing step between the database and the web page. It reads tables, computes our measures and the model, and writes prepared result tables back.
API / endpoint / JSON	The API is the set of web addresses the page calls. Each address is an endpoint. JSON is the plain-text format of the answer.
Champagne Index	Our own measure of attacking output: recorded goals plus assists, divided by the number of matches the source reports for that club-season. It is a proxy, not an official statistic.
Target and feature	The target is what the model tries to predict — here the change in the index in the next consecutive season. A feature is an input the model uses to predict it.
Chronological holdout	We train the model on earlier seasons and test it on later seasons it has never seen. A random split would mix the future into the training data and flatter the result.
Baseline	The simplest possible prediction, used for comparison. Ours predicts no change at all. A model is only useful if it beats the baseline on the same test rows.
R^2 and MAE	R^2 is the share of variation the model explains, from 0 to 1 — ours is 0.127, which is weak. MAE is the average absolute error in index units — ours is 0.4856 against a baseline of 0.5114, about 5% better. Neither is a percentage of correct answers.
Correlation vs. causation	A correlation (r) measures how two numbers move together, from −1 to +1. It cannot show that one causes the other. Every result in this project is a correlation, including the red-card and spending findings.
Proxy	A measurable stand-in for something we cannot measure directly. Our shots-on-target estimate is a proxy for chance quality; it is not xG, which the data source does not contain.
Poisson rating	A simple statistical model of how many goals each team scores and concedes, fitted to one league-season. It describes that season; it does not forecast new matches.
SHAP	A method that attributes a model's prediction to its individual inputs, so we can report which feature mattered most without relying on the model's internal importance score.

Practice

1. Explain the difference between 422 and 503 to someone who has never used an API, in one sentence each.
2. Say out loud why “no recorded meeting” and “0–0” must look different on the screen.
3. Call /api/head-to-head for two clubs from the same city and verify that reversing the clubs swaps goals for and goals against.

Questions for your part

Why have an API instead of plotting raw tables directly?

The API exposes small prepared outputs under an explicit league and season. This avoids moving millions of raw rows to the browser, keeps SQL and validation on the server and separates presentation from processing.

How are head-to-head goals calculated?

For each completed meeting, the selected club’s perspective determines whether the home or away score is goals for. Reversing the compared clubs swaps their totals. Wins, draws, losses and points are derived from those actual scores.

Does no recorded match mean a 0–0 draw?

No. No coverage, no recorded meetings and a completed 0–0 draw are different states. The API reports enrichment availability and the interface shows an explicit message instead of inventing a score.

What prevents invalid scenario inputs?

The API rejects non-finite numbers, checks bounds and requires attacking spend not to exceed total spend. The browser also clamps the sliders. Server-side validation remains necessary even when the UI enforces the same rule.

What do HTTP 422, 404 and 503 mean here?

422 indicates invalid input, such as selecting the same club twice. 404 means the requested club-season scope is unavailable. 503 indicates temporary data/model unavailability or an incomplete application output state.

What do your tests prove?

Regression tests cover known failure cases, live checks validate the prepared outputs and API contracts, and browser checks exercise actual flows and mobile layout. They provide scoped evidence, not a guarantee that every possible bug is absent.

What is the main result in one sentence?

The system is reproducible on the measured setup, while historical transfer spending alone has almost no linear relationship with next-season index change and the richer model adds only modest predictive value.

What is the most important limitation?

The data are retrospective and unevenly covered, the index is a proxy, and associations are not causal effects. Source licence and independent team/laptop validation also remain explicit unresolved requirements.

How do goals for, goals against and goal difference work in the comparison?

GF and GA are season totals rebuilt from completed home/away match scores. GD is GF − GA. They come from the optional football-data.co.uk match enrichment, not from the Champagne Index player totals.

How are attacking and defensive ranks calculated?

Within one league and season, attack rank orders goals scored per match from highest to lowest; defense rank orders goals conceded per match from lowest to highest. SQL RANK gives tied rates the same rank and leaves a gap after ties. They describe scores, not tactics or chance quality.

Do we have xG or can we estimate it?

The source has no provider xG field. The comparison instead shows observed shots and shots on target. A separate estimate multiplies shots on target by the league-season average goals-per-shot-on-target rate; it is an experimental volume proxy with no shot-location or chance-quality input, not xG.

What is the difference between the Champagne Index and attack rank?

The Champagne Index is recorded player goals plus assists divided by a Transfermarkt matches proxy for a club-season. Attack rank uses actual completed league scorelines, sums team goals and ranks goals per match within the league-season. Their sources, coverage and units differ.

How did you check the goalkeeper defensive totals?

We compared Transfermarkt goalkeeper goals-conceded totals with the independent match-result totals on 388 adequately covered top-five club-seasons. They match exactly in 366 (94.3%). Clean sheets match only 269 (69.3%), so the scoreline source remains the basis of league ranks.

What does the defensive-spend analysis show?

The Pearson correlation between defender transfer fees and next-season change in the goalkeeper-source conceded-goal rate is r=0.004047, p=0.426997, n=38,531. The sample does not support a useful linear association. It is descriptive, not causal. A separate match-level check finds teams with a red card concede 0.341 more goals on average in the same match, but match context confounds that association.

Why keep the player-source defensive values separate from scoreboard goals?

The sources use different records and denominators. Goalkeeper conceded totals agree with scorelines in 94.3% of adequately covered seasons, while clean sheets agree in 69.3%. The UI therefore uses match scores for GF, GA, GD and ranks; player-source totals remain a quality cross-check. Shots and corners are also observed match statistics, not chance-quality measures.

Is the shot-based estimate the same as xG?

No. It multiplies a team’s shots on target by the league-season average goals per shot on target. It has no shot-location, distance, body-part, pressure or goalkeeper-position input. It is a rough volume benchmark and must be called an SOT-based proxy, not provider xG.

What does the Poisson attack/defense rank tell us?

The model fits attack, defense and a shared home-advantage term to the selected league-season scores. Larger fitted attack and defense parameters mean stronger estimated attack and defense. Its ranks are descriptive and in-sample; the separate chronological RandomForest evaluation is the project’s held-out prediction check.

Did the added defensive features improve the model?

On the same chronological split, R^2 moves from 0.1271 with the five-feature baseline to 0.1502 after adding goalkeeper conceded rate, discipline and net spend; MAE moves from 0.4856 to 0.4788 on 5,585 held-out rows. The gain is modest, and it does not isolate a causal effect of defense.

Do red cards cause more goals to be conceded?

The local sample shows a same-match difference of +0.341 conceded goals per team-match (1,619 sent-off vs 16,195 others; Welch $p < 0.0001$). This is observational: stronger opponents, match state and timing can affect both red cards and goals. It is not a causal estimate.

Phrasing to avoid

- Missing data is the same as zero.
- The sliders in the Lab show what a transfer would cause.
- The tests prove the application has no bugs.

Exam day

Date	Format	What is required
12 November 2026 CIDS Lecture 15, pp. 37–39	Checkpoint · 5 minutes	One end-to-end demo, one visible result and the first measured full-run time. A polished interface is not required.
20 / 26 November 2026 CIDS Lecture 15, pp. 37–39	Final defense · 15 minutes + 5 minutes of questions	Every real participant speaks. Suggested split: question and data 3 min, architecture 2, live demo 3, measurements 3, problems 2, contributions 2.

Machine and materials

1. Thirty minutes ahead: run `docker compose up -d --build`, then open /api/health and wait for `ready=true` with the loader and pipeline finished.
2. Open four tabs in advance: Overview (Serie A 24/25), the Inter / Milan comparison, the Lab, and the Project page with the numbers.
3. Open the offline study site `defense/site/index.html` and `defense/site/downloads/Presentation.pdf`: both work without a network and without the database.
4. Check that the screenshots in `defense/site/assets/*-desktop.png` are present. They are the fallback if Docker does not come up.
5. Turn off notifications, connect the charger, test screen mirroring and increase the browser font size.
6. Decide in advance who drives the demo and who watches the clock. The timer is built into `defense/site/presentation.html` (N for notes, F for fullscreen).

How to answer questions

- Answer in three steps: one short sentence, then the evidence (a file or a number), then the limitation.
- Give every number with its sample and unit: “r = −0.79 across 388 club-seasons, within the same season”, not “the correlation is high”.
- If you do not know, say where the answer lives instead of guessing: “that is in docs/MEASUREMENTS.md, the memory sampling method is described there”.
- If the question belongs to someone else's part, hand it over in one sentence rather than approximating their area.
- Never turn an association into a cause. Say “accompanies” or “is associated with”, never “causes” or “improves”.
- If asked about a weak point, answer it first and directly. An acknowledged limitation is stronger than a defended inaccuracy.

Uncomfortable questions: answer them first

Topic	Question	Answer and evidence
Licence of the primary source	Is your data openly licensed?	We pinned the source revision and verified headers and checksums, but there is no licence file, so we did not establish an explicit reuse licence. We document it as unresolved and we do not redistribute the raw data. Public downloadability is not a licence. `docs/SOURCES.md, docs/LICENCE_DECISION.md`
Four contributors	Who did what in this project?	Describe only the work you can open and explain on screen. Git identities and role cards are not proof of independent contribution, and the repository says so. `docs/WORK_PACKAGES.md, README.md`
Reproduction on another machine	Did someone else reproduce it?	The run is reproducible on the measured machine, and we also rebuilt it from a clean clone. An independent fresh checkout on another person's laptop has not been recorded yet, so we do not claim it. `docs/FRESH_CHECKOUT.md, docs/MEASUREMENTS.md`
xG and proxies	Is this xG?	No. The source has no provider xG field. We show observed shots and shots on target, and a shots-on-target conversion proxy that we label experimental. The Poisson attack and defense ratings describe the season they are fitted on. `docs/GLOSSARY.md, pipeline/proxy_xg.py, pipeline/power_ratings.py`

> A role is a study responsibility. Do not claim authorship or a verification you did not perform. Show exactly the changes and commands you ran yourself.